Defense pack — MLE-AI-201 final project (bina.az, from-scratch tree + Pegasos SVM)

Files: `presentation/slides_defense.pptx` (editable, speaker scripts in notes) · `presentation/slides_defense.pdf` · this document. Everything below was checked against the repository at tag `v1.0-final` (`results/results.json`, `results_run.log`, `src/`, `tests/`).

A. Project audit

A1. Verified facts (evidence in the repository)

Area	Fact	Evidence
Dataset	Kaggle `binaaz-sale-project`, 100,775 rows, 51 columns, every row AZN (asserted in code)	`data/README.md`, `src/data_prep.py` (currency assert), `results_run.log`
Cleaning	100,775 → 64,454 (latest scrape per URL) → 58,452 (residential) → 58,110 (fixed bounds) → 57,421 (repost duplicates)	`results.json: cleaning_log`
Leakage	`unit_price`, `total_price` dropped; identifiers, text, views/vip/featured dropped	`data_prep.LEAKAGE_COLUMNS`, `DROP_COLUMNS`
Features	55 = 9 numeric + 12 binary flags + 34 one-hot; district vocabulary fitted on train rows only	`make_features(df, train_idx)`; verified by counting unique values per column
Split	Stratified on log-price deciles, seed 42: 40,197 / 8,612 / 8,612; median log-price 12.2453 in all three; premium 49.9 / 49.8 / 49.8 %	`stratified_split_indices`, log §2, test `test_stratified_split_balances_price_and_tier`
Tier label	threshold = train median = 208,000 AZN	`make_tier_label`, log §2
Decision tree	Gini / entropy / MSE; vectorised split search (cumulative counts, Σy, Σy²); sklearn-weighted impurity decrease; `max_depth`, `min_samples_split`, `min_samples_leaf`, `min_impurity_decrease`,	`src/decision_tree.py`; tests pin identical split to sklearn on toy data and that each hyperparameter changes the tree

Area	Fact	Evidence
	`max_features`; importances; `predict_proba`	
Tree results	Task A test RMSE(log) 0.2289 (sklearn 0.2287); CV 0.2298 ± 0.0022 vs 0.2295 ± 0.0021. Task B F1 0.9228 (sklearn 0.9233); CV 0.9226 ± 0.0022 vs 0.9224 ± 0.0026. Chosen: depth 18 / leaf 10 (A), Gini depth 16 / leaf 5 (B)	log §3, §4, §6, §7
SVM	Pegasos mini-batch (64), η = 1/(λt), unregularised bias, three LR schedules (`pegasos`, `inv_sqrt`, `constant`), `n_epochs`, objective tracking, support vectors, margin 2/‖w‖; RFF (D = 300)	`src/svm.py`; tests: converges, ≈ LinearSVC, all schedules train
SVM results	λ sweep: margin 0.028 → 0.844, best λ = 1e-3; epochs plateau ≈ 10; schedules: pegasos F1 0.897 / J 0.291, constant 0.893, inv_sqrt 0.886 (margin 0.91); final ‖w‖ 3.79, margin 0.53, SVs 11,917 / 40,197; test F1 0.8971 (SGD 0.8989, LinearSVC 0.8934); CV 0.8988 ± 0.0016 vs 0.9000 ± 0.0012; RFF γ = 0.01, F1 0.8491; SVC-RBF (15k) 0.8449	log §5, §6, §7
Metrics	RMSE, MAE, R^2, confusion matrix, precision/recall/F1, rank-based ROC-AUC, k-fold — all NumPy; test asserts equality with sklearn to 1e-9	`src/evaluate.py`, `tests/test_models.py`
Error analysis	58.5 % (tree) / 61.9 % (SVM) of misclassifications within ±15 % of threshold; decile error 13.2 % (cheapest), 6.7–8.3 % (mid), 16.0 % (top); Mərdəkan 16.1 %; k-means segments 8.1–20.8 %	`results.json`
Bonuses	Ridge (normal eq.) = sklearn to 1e-15, RMSE 0.3006; LinearSVR (sub-gradient, ε = 0.2) = sklearn LinearSVR 0.3044; RandomForest ×30: 0.2211 vs sklearn 0.2166; k-means/PCA segments	log §9, `src/baselines.py`, `src/ensemble.py`
Reproducibility	`python -m src.run_all` runs 10 tests then everything (590 s); pinned requirements; seed 42; results.json + tables.md	`README.md`, `results_run.log`
Repo	github.com/thuseynowa/bina-ml-final, tag `v1.0-final`, 13 commits by module	`git log`

A2. Interpretations (presented as such on the slides)

- **Why linear beats RBF**: 46 / 55 features are binary and the tier boundary is near-linear in area + centrality, so the RBF geometry adds variance. Supported by two independent RBF implementations failing alike (0.849 vs 0.845) — but not proven; a larger D or finer γ grid was not tried.
- **Why the forest is worse in AZN but better in log**: retransformation bias of exp(E[log y]) — a known effect; not measured with a smearing correction.
- **Tail errors are due to unobserved attributes** (condition, finish, view): plausible, consistent with the unused description text, not tested.
- **Residual tree-vs-sklearn gap = tie-breaking + float midpoints**: consistent with the toy test that shows identical splits when no ties exist; not traced split-by-split on the real data.

A3. Missing or weak evidence (and whether it matters)

Gap	Essential?	Handling
Git history: the first two commits carry most of the code; per-member commits are being added now	Yes for the rubric's contribution check	Each member commits to their own module (instruction sheet already distributed); the slides list contributions as in `contribution_report.md`
SVC-RBF baseline trained on a 15k subsample, not matched in data size	Minor	Stated on the slide; matched baselines are SGDClassifier and LinearSVC
SVR best ε at the edge of its grid (0.2)	Minor (bonus)	Stated in Q&A; cheap to extend
No smearing-corrected AZN metrics	Minor	Listed as limitation and next step
No group split by building (flats in one building share coordinates)	Minor	Listed as future work; it is not leakage, it is legitimate signal
PR-AUC not computed (statement allows ROC-AUC **or** PR-AUC)	No	ROC-AUC + confusion matrix satisfy the statement; a PR-AUC task is assigned to Nihat as an extra commit

---

B. Presentation blueprint (16 slides, ≈ 14 min + questions)

#	Speaker	Slide	Narrative purpose
1	Tahmina	Title	Who we are; the one-sentence question
2	Tahmina	One dataset, two tasks, two models	Problem, scope, metrics, golden rule; preview of the headline
3	Tahmina	Cleaning removed 43 %	The cleaning funnel with row counts; EDA figure
4	Tahmina	Leakage and features	What was dropped and why; what 55 features are; train-only encoding
5	Tahmina	Stratified split and protocol	Split diagram, balance table, the no-leakage rules, one-command reproduction → hand-over
6	Elshan	Vectorised split search	How the tree works: objective, 4-step flow, stopping rules, complexity

#	Speaker	Slide	Narrative purpose
7	Elshan	Bias–variance	Depth / leaf sweeps; chosen configuration on validation
8	Elshan	Tree vs sklearn	Matched comparison with CV uncertainty; where the residual comes from → hand-over
9	Alish	Pegasos formulation	Objective, update rule, bias, scaling, SVs, RFF choice; objective curve
10	Alish	λ / epochs / schedule	The three solver knobs the statement requires; margin–fit trade-off
11	Alish	Margin, SVs, linear vs RBF	What was learned; Pegasos vs twins; honest RBF result → hand-over
12	Nihat	Full results tables	Both tasks separately, uncertainty, timings
13	Tahmina	Error analysis	Threshold band, tails, periphery; importances agree
14	Nihat	Bonuses	Ridge + SVR, forest curve, k-means segments
15	Nihat	Engineering + limitations	Reproducibility and tests; six limitations stated plainly
16	Nihat	Conclusion + contributions	Takeaway, next steps, who did what, Q&A

Narrative arc: question → fair protocol → two mechanisms → matched evidence → where it fails → what we would do next.

D. Design specification

- **Canvas**: 16:9 wide (13.33 × 7.5 in). Margins 0.6 in. One message per slide; the title is the message (sentence case, no final period).
- **Palette** (one dominant, one accent, figure-matched): ink navy `1B2A41` (titles, dark slides), white `FFFFFF` (content background), panel grey `F2F4F7` (cards, takeaway band), muted `5B6778` (captions), accent amber `E69F00` (primary accent, matches the figures' orange), blue `0072B2`, green `009E73`, vermilion `D55E00` — the Okabe-Ito colours already used in every figure, so charts and slides read as one system.
- **Typography**: Cambria (headings, 26 pt content titles / 40 pt dark slides, large stat numbers 30 pt), Calibri (body 12–15 pt, captions 10–12 pt). Both ship with Office and render true-to-width.
- **Layouts**: `DARK` (title + conclusion), `CONTENT` (title placeholder, footer "MLE-AI-201 · Final project · bina.az", slide number). Content composed from four components: stat card, text card, native table (navy header row, light grey rules), fitted project figure. Every content slide ends with a grey **Takeaway** band.
- **Figures**: only figures generated by `run_all.py` (`report/figures/*.png`), inserted at native aspect ratio; no decorative imagery, no icons, no accent bars.
- **Tables**: native PowerPoint tables (editable), 11–12.5 pt, bold for "ours" headline numbers only.
- **Consistency rules**: titles at the same position on every content slide; one chart-label style (inherited from matplotlib); bullets left-aligned; stat cards 1.35 in tall; cards share corner radius 0.08 in.

- **Accessibility**: all text ≥ 10 pt, contrast ≥ 7:1 on white and on navy, colour never the only carrier of meaning (tables carry the numbers).

G. Final quality audit

Checked: every number on the slides against `results/results.json` and `results_run.log`; every mechanism claim against `src/`; every test claim against `tests/test_models.py`; contributions against `contribution_report.md`.

Corrected during the audit

1. The report and the Beamer slides said "38 of 55 features are binary". Counting unique values in the real design matrix gives **46 binary, 9 numeric**. Fixed in `report/report.tex`, `presentation/slides.tex`, the new deck and the scripts; PDFs rebuilt.
2. Slide 4 originally listed "numeric (10) / binary (8)"; now 9 / 12 / 34 (= 55).

Unresolved issues to state honestly if asked

- SVC-RBF baseline uses a 15k subsample (not matched in data size).
- Depth-18 vs 16–24: the validation curve is flat; the chosen depth is the arg-min, not a statistically distinct optimum.
- SVR ε at grid edge; RFF γ grid coarse, D = 300.
- Git history is front-loaded; per-member commits are being added before the deadline.
- "Linear beats RBF because features are binary" is an interpretation, not a demonstrated cause.

Consistency: slides ↔ scripts ↔ report ↔ results.json agree on all headline numbers (0.2289 / 0.2287; 0.9228 / 0.9233; 0.8971 / 0.8989; 0.8491 / 0.8449; 0.2211 / 0.2166; 0.3006; 0.3044; 58.5 % / 61.9 %; 40,197 / 8,612 / 8,612; 208,000).

Defense guide — slide content, speaker scripts, examiner Q&A

Deck: `presentation/slides_defense.pptx` (speaker scripts are also in each slide's notes). Total ≈ 14 minutes.

Speaking plan

Speaker	Slides	Time
Tahmina	1, 2, 3, 4, 5, 13	5:20
Elshan	6, 7, 8	3:00
Alish	9, 10, 11	3:10
Nihat	12, 14, 15, 16	3:20

Slide 1 — Baku Apartment Prices from the Inside

Speaker: Tahmina · **time:** 0:30

Script

Good afternoon. We are Alish, Tahmina, Nihat and Elshan. Our project asks one question: can two classical models written from scratch in NumPy — a decision tree and a support-vector machine — stand next to scikit-learn on real, messy apartment listings from bina.az? I will present the data and the experimental protocol, Elshan the tree, Alish the SVM, and Nihat the benchmark, bonuses and conclusions. I will also come back for the error analysis.

Slide 2 — One real dataset, two learning tasks, two models we had to build ourselves

Speaker: Tahmina · **time:** 1:00

Takeaway on slide: Same features, same split, matched hyperparameters — then the comparison is fair and the number means something.

Script

Our data comes from bina.az, the largest real-estate platform in Azerbaijan — one hundred thousand scraped listings. From this one dataset we built two tasks. Task A is regression: we predict the logarithm of the price, because prices have a heavy tail — a few very expensive apartments would otherwise dominate the error. Task B is classification: is a listing "premium" or "standard", where premium means above the median price of the training set, 208 thousand manat. The golden rule of the project: both models are our own NumPy code. Scikit-learn is used only as a baseline, with exactly the same features, the same split and matched hyperparameters. The headline, which Nihat will show in detail: our models land within about 0.002 of their scikit-learn twins on both tasks.

Likely examiner questions

- **Why predict log(price) instead of price?** Prices are heavy-tailed (Fig. EDA a/b). On the log scale the distribution is close to normal and the squared error becomes relative: a 10% miss on a cheap flat and on a villa count the same. We still report AZN metrics after back-transforming.
- **Why a median split for the tier?** It gives balanced classes by construction and the threshold is defined from the training split only, as the statement requires. We note in the limitations that this makes the label noisy near 208k.

Slide 3 — Cleaning removed 43% of rows — every rule is fixed, documented and reproducible

Speaker: Tahmina · **time:** 1:00

Takeaway on slide: Rules, not statistics: a fixed bound is a decision made before any split exists, so it cannot leak.

Script

The raw data is a real scraped dump, so cleaning was a large part of the work, and every rule is code in data_prep.py. First, the scraper visited the same listing many times, so we keep the latest version of each listing URL — that removes thirty-six thousand rows. Second, we keep only residential categories; offices and land plots quote areas in different units and are a different market. Third, many numbers arrive as Azerbaijani text — "145 square metres", "floor 7 of 9" — and we parse them. Fourth, outliers: we use fixed bounds — price between ten thousand and three million manat, area between fifteen and eight hundred square metres, coordinates inside Azerbaijan. We deliberately avoid quantiles, because a quantile is a statistic of the whole dataset including the test rows. Missing values get a constant plus an indicator column — again, nothing is fitted. The figure shows what we are left with: a heavy-tailed price, a near-normal log-price, and area as the strongest single predictor.

Likely examiner questions

- **Why fixed bounds rather than IQR or percentile filters?** A percentile is computed on the full data, including the test rows, so it leaks evaluation information into preprocessing. A fixed bound is a decision taken before the split and documented; it removes 342 rows.
- **Why dedupe by URL and then again by content?** The first pass removes repeated scrapes of the same page; the second removes the same flat re-posted under a new URL with identical price, area, floor and coordinates (689 rows). Leaving those in would put near-copies in train and test.

Slide 4 — Two columns were arithmetic functions of the target — dropped before any model saw them

Speaker: Tahmina · **time:** 0:50

Takeaway on slide: The statement promised to check for leakage; our feature matrix contains nothing computed from the answer.

Script

The most important slide of my part. The dataset contains two columns, unit_price and total_price. Unit price is price divided by area; total price equals the price in 99.99 percent of rows. They are computed from the target. If you train on them, validation looks perfect and the model is useless on a new listing. We dropped both, and we also dropped identifiers, free text and popularity counters such as views — those are consequences of the listing, not properties of the apartment. What remains are 55 features: numerics like area and the distance to the city centre, binary flags, and one-hot encodings for the building category and the thirty most frequent districts. One detail we are proud of: the list of thirty districts is chosen from the training rows only, so even the encoding does not see validation or test data.

Likely examiner questions

- **How did you confirm unit_price and total_price are leakage?** We checked it in pandas before modelling: total_price == price in 99.99% of rows and unit_price × area ≈ price. Both are deterministic functions of the target.
- **Why drop views / vip / featured? They are real columns.** They are generated after the listing exists — an expensive flat gets promoted and viewed more. They would not be available for a new listing and would act as soft leakage.
- **Would ordinal encoding of districts have worked for the tree?** A tree can split an ordinal code, but the ordering would be arbitrary, so it would need many splits to isolate one district. One-hot gives each district a clean threshold; for the SVM one-hot is required anyway.

Slide 5 — Stratified 70/15/15 split; every statistic is learned from the training rows only

Speaker: Tahmina · **time:** 1:00

Takeaway on slide: We cannot stratify on the tier label — it only exists once train and its median exist — so we stratify on price deciles, which matches both tasks.

Script

Now the protocol. We use a seeded 70-15-15 split that is stratified on deciles of log-price: each decile is shuffled and cut separately, so all three parts have the same price distribution — the median log-price is identical to four decimals. This stratifies the regression task directly, and the classification task by construction, because the premium label is "above the train median", a cut inside the fifth decile; the premium share comes out at 49.9, 49.8, 49.8 percent. Two rules we followed strictly: the threshold, the scaler and every encoding choice come from the training rows only, and all hyperparameters are chosen on validation. The test set is used once per final model. Everything is reproducible with one command, which first runs ten sanity tests against scikit-learn on toy data and then regenerates every number and figure you will see. Elshan will now open the decision tree.

Likely examiner questions

- **Why not stratify on the tier label directly?** The label does not exist before the split — it is defined by the training median. Stratifying on log-price deciles gives the same balance and additionally matches the regression target distribution.
- **Could near-duplicate flats still sit in both train and test?** Exact reposts are removed. Different flats in the same building share coordinates and district, which is legitimate signal, not leakage; a group split by building would be a stricter alternative and is in our future work.
- **Why 70/15/15 and not k-fold for everything?** Hyperparameter sweeps on 40k rows with our Python tree are expensive; a fixed validation split keeps the sweeps tractable. We then add 5-fold CV on the pooled train+val for the final configurations to report mean ± std.

Slide 6 — Our tree scores every candidate threshold of a feature in one vectorised pass

Speaker: Elshan · **time:** 1:10

Takeaway on slide: The algorithm is small: ~250 lines of NumPy reproduce the library once the objective and the weighting are implemented exactly.

Script

Thank you, Tahmina. The tree is a CART-style binary tree with a scikit-learn-like fit, predict and predict_proba API. The core is the split search. The naive approach loops over every feature and every threshold and recomputes the impurity each time — far too slow for forty thousand rows in Python. Instead, for each feature we sort the samples once per node and walk the sorted order with cumulative sums: cumulative class counts for classification, cumulative sums of y and y-squared for regression. From these prefix sums the impurity of every possible left and right child falls out in a single NumPy expression, and we pick the largest weighted decrease. The threshold is the midpoint between the two neighbouring values, exactly as scikit-learn does. We support Gini, entropy and MSE, four stopping rules that are each covered by a unit test, and

feature importances as the accumulated impurity decrease. The complexity is the same as the library's; the twelve-times wall-clock gap is Cython versus NumPy, not the algorithm.

Likely examiner questions

- **How do you compute the variance of every prefix without a loop?** With S1 = Σy and S2 = Σy² over the prefix of length n: var = S2/n − (S1/n)². Both are cumulative sums, so all prefixes are scored at once.
- **How do you handle ties in the feature values?** Candidate positions are only where consecutive sorted values differ, so a threshold never falls between equal values. Ties in gain are broken by the first feature in column order; sklearn samples feature order randomly, which explains the last-decimal differences.
- **Why entropy in bits and not nats?** To match scikit-learn's criterion='entropy' numerically so the comparison is like-for-like; the chosen split is invariant to the base anyway.
- **Does min_impurity_decrease work the same as sklearn's?** Yes — we apply the same weighted rule N_t/N · Δ ≥ threshold, and a test asserts it actually prunes.

Slide 7 — Validation error bottoms out at depth 18 — deeper trees only memorise the training set

Speaker: Elshan · **time:** 0:55

Takeaway on slide: Classic bias–variance: the train–validation gap widens with depth; the chosen configuration sits at the knee, not at the edge of the grid.

Script

This is the bias-variance story. On the left, training RMSE on log-price falls monotonically with depth, from 0.44 down to 0.14, while validation error flattens at depth 18 and then creeps up to depth 30 — the tree starts memorising. We chose depth 18 on validation, then swept min_samples_leaf at that depth and found a clean U-shape with the optimum at ten. For the classification task Gini and entropy are nearly identical, Gini slightly ahead, with the best depth at sixteen. All of this is chosen on the validation split; the test set was not touched until the final models were fixed.

Likely examiner questions

- **The validation curve is almost flat from 16 to 30 — is depth 18 really better?** No, within noise it is not; the curve is flat. We pick the minimum, but any depth 16–24 with leaf 10 gives the same test error. The important point is that deeper does not help, and training error keeps falling, which is the overfitting signature.

- **Why sweep depth first and then leaf size, not a full grid?** A full grid of 14 depths × 6 leaf sizes with a Python tree would take over an hour per task. The two-stage sweep is cheap and the interaction is weak; the leaf sweep at the chosen depth refines the optimum.

Slide 8 — With matched hyperparameters our tree reproduces scikit-learn to the third decimal

Speaker: Elshan · **time:** 0:55

Takeaway on slide: The library's edge is engineering (Cython), not statistics — the numbers agree within cross-validation noise.

Script

Here is the comparison that the project is built around. With the same depth, leaf size and criterion, our tree and scikit-learn's agree on the test set to the third decimal on both tasks, and five-fold cross-validation shows the agreement is systematic: the standard deviation across folds is about 0.002, ten times the difference between the implementations. Where does the residual come from? Tie-breaking: when several splits have equal gain we take the first feature in column order, scikit-learn samples features in random order; and midpoint thresholds can differ in the last floating-point bits. The one real gap is speed — six seconds versus half a second — and that is Cython versus NumPy, not the algorithm. Alish will now show the SVM.

Likely examiner questions

- **Is a 0.0002 RMSE difference meaningful?** No. The 5-fold standard deviation is 0.002, so the implementations are indistinguishable statistically. We report it to be transparent, and we explain its source.
- **Could you make your tree as fast as sklearn's?** Not in pure Python; the remaining cost is the per-node Python loop over features and recursion. Numba or Cython on the split search would close most of the gap.

Slide 9 — Pegasos: the soft-margin SVM solved by sub-gradient descent — no QP, no SMO

Speaker: Alish · **time:** 1:10

Takeaway on slide: RFF over kernelised Pegasos: at n ≈ 40k, dual coefficients make every prediction O(n); RFF stays linear-time with one solver.

Script

The SVM is trained with Pegasos. Instead of solving the dual quadratic programme, we minimise the primal directly: the regularised hinge loss, lambda over two times the squared norm of w plus the average hinge. Each step draws a mini-batch of 64 examples; for the examples inside the margin we move w towards y-times-x, and in every step we shrink w by one minus eta-lambda — that is the regulariser. The step size is one over lambda-t, the schedule from the Pegasos paper. The bias gets the same averaged sub-gradient but no shrinkage. Because the hinge loss is scale-sensitive we standardise features with training statistics. After training we read off the support vectors — the points on or inside the margin — and the margin width two over the norm of w. On the right you see the objective decreasing for three values of lambda. For a non-linear boundary we chose random Fourier features: a random cosine map whose inner products approximate the RBF kernel, so the identical linear solver runs on the transformed features in linear time.

Likely examiner questions

- **Why is the bias not regularised?** Regularising b would penalise shifting the hyperplane, which has nothing to do with margin width. Standard formulations keep b free; folding a constant-1 feature into x (the alternative in the statement) would regularise it slightly — we chose the explicit, unregularised bias and say so.
- **Why does 1/(λt) make sense as a step size?** The objective is λ-strongly convex; the Pegasos analysis shows that with $\eta_t = 1/(\lambda t)$ the sub-gradient method converges at rate $\tilde{O}(1/(\lambda T))$ with no tuning of the step size. The objective curve confirms it.
- **Why mini-batches of 64 rather than one example?** Variance reduction: averaging 64 sub-gradients gives a smoother objective trace at the same cost per step in NumPy. The update rule is identical with the mean replacing the single example.
- **Why RFF and not the kernel trick?** Kernelised Pegasos keeps a coefficient per training example, so prediction costs O(n) kernel evaluations and training $O(n^2)$ — infeasible at 40k. RFF costs O(nD) with D = 300 and reuses the linear code unchanged.

Slide 10 — λ sets the margin–fit trade-off; 1/(λt) converges fastest; F1 plateaus after ~10 epochs

Speaker: Alish · **time:** 1:10

Takeaway on slide: All three solver knobs the statement asks for — λ, epochs, LR schedule — are exposed, and each visibly changes the margin or the metric.

Script

The statement asks us to expose three solver hyperparameters and show that each matters. Lambda first: as it grows from ten to the minus six to ten to the minus two, the margin widens thirty times — stronger regularisation means a wider margin — while validation F1 peaks at ten to the minus three and then drops as the model under-fits. Epochs: one epoch is 629 mini-batch steps; F1 is essentially at its plateau after ten epochs, so our thirty thousand steps are generous. The schedule: the Pegasos one-over-lambda-t schedule reaches the lowest objective and the best F1; a constant step is close; one-over-root-t shrinks w too slowly, so it ends with a wider margin but a worse-placed boundary. Finally the RFF bandwidth gamma: 0.01 is best, and larger values over-fit sharply.

Likely examiner questions

- **How does λ relate to C in sklearn?** C = 1/(nλ). With n = 40,197 and $\lambda = 10^{-3}$, C ≈ 0.025 — the value we pass to LinearSVC and SVC in the benchmark.
- **Why does a wider margin not always mean better F1?** The margin is 2/‖w‖, and λ shrinks w regardless of the data. Past the optimum, the hinge term is under-weighted and the hyperplane stops separating the classes well — larger margin, more violations.
- **Why did $\eta_0/\sqrt{t}$ give a wider margin but lower F1?** With $\eta_t = \eta_0/\sqrt{t}$ the shrink factor $(1 - \eta_t \lambda)$ is far weaker than Pegasos's, so ‖w‖ stays small (wide margin) but w has not converged to the optimum direction after 30k steps.

Slide 11 — 30% of training points are support vectors — the tiers interleave at the threshold; linear beats RBF here

Speaker: Alish · **time:** 0:50

Takeaway on slide: A margin you can see: 30% of points inside it tells us the classes genuinely overlap near 208k — the label, not the model, is the limit.

Script

What did the SVM learn? The weight vector has norm 3.79, so the margin is 0.53 in standardised units, and almost twelve thousand of the forty thousand training points — thirty percent — are support vectors, on or inside the margin. That is a lot, and it is telling us something real: the two tiers genuinely interleave around the 208-thousand threshold. Against scikit-learn, Pegasos matches SGDClassifier with hinge loss and the same lambda within cross-validation noise. The surprising result is that both non-linear models lose to the linear ones by five F1 points — ours with random Fourier features and scikit-learn's SVC with an RBF kernel. Our reading: after one-hot encoding, 46 of 55 features are binary, and the premium boundary is close to linear in area and distance to the centre, so the RBF geometry mostly adds variance. The fact that two independent RBF implementations agree supports that. And note the cost: the exact RBF SVC needed

sixteen seconds to fit on a fifteen-thousand-row subsample and eleven seconds just to predict. Nihat will now put everything side by side.

Likely examiner questions

- **Could a larger D or a finer γ grid close the RFF gap?** Possibly part of it — D = 300 is small and the γ grid is coarse; we list this as a limitation. But sklearn's exact RBF SVC fails the same way, which suggests the boundary is simply near-linear in these features.
- **Why is SVC trained on a subsample — is that a fair comparison?** It is not matched in data size, and we say so: exact kernel SVMs are $O(n^2)$ and 40k rows did not fit our time budget. It is included to show the cost that motivates RFF, not as the primary baseline; the matched baselines are SGDClassifier and LinearSVC.
- **Does LinearSVC with C = 1/(nλ) optimise the same objective?** The same loss and regulariser, but LinearSVC uses squared hinge by default and a coordinate-descent dual solver, and it regularises the bias; hence the slightly lower F1. SGDClassifier(hinge, alpha = λ) is the closer twin.

Slide 12 — On both tasks the from-scratch models land within 0.002 of their scikit-learn twins

Speaker: Nihat · **time:** 1:00

Takeaway on slide: Regression and classification are reported separately with task-appropriate metrics; no single number declares a winner.

Script

Here is the full benchmark — all metrics implemented from scratch in evaluate.py, and a test asserts they equal scikit-learn's to one-e-minus-nine. Task A on the left: our tree and scikit-learn's agree to the third decimal on RMSE, R-squared and MAE in manat. The bonus forest improves the log-scale RMSE; the two linear bonus models trail by seven hundredths. Task B on the right: tree versus tree, Pegasos versus SGDClassifier, each within two thousandths of F1. Below the tables: five-fold cross-validation on the pooled train-plus-validation data, with the tier threshold and the scaler re-fitted inside every fold, so no fold leaks. The standard deviations — about 0.002 — are larger than every ours-versus-sklearn difference. And timings: the honest cost of pure NumPy is a ten-times slower tree.

Likely examiner questions

- **Why report both log and AZN metrics?** The model is fitted on log-price, so log metrics describe the optimisation target; AZN metrics are what a user cares about. Both are in the report; the AZN numbers come from exponentiating predictions, which we discuss as a limitation.

- **How was the CV done without leaking the tier threshold?** Inside each fold the training median is recomputed from that fold's training rows, labels for the validation fold use it, and the scaler is refit — see cross_validation() in run_all.py.
- **Why is the forest better on log RMSE but worse on AZN RMSE?** exp(prediction of log price) estimates the conditional median, not the mean, so expensive listings are under-predicted; the forest averages more and the effect is stronger in AZN. A smearing correction is the standard fix.

Slide 13 — Errors cluster in three places: at the tier threshold, in the price tails, and in the periphery

Speaker: Tahmina · **time:** 1:00

Takeaway on slide: The remaining error is explained, not random: label noise at the cut, unobserved attributes in the tails, and sparse peripheral districts.

Script

Let me come back for the error analysis — where and why the models fail. For classification: fifty-eight percent of the tree's mistakes and sixty-two percent of the SVM's are listings whose true price is within fifteen percent of the 208-thousand threshold. For those the label is almost a coin flip, so most of the remaining error is in the framing, not in the model. For regression, the left panel: median relative error is seven to eight percent through the mid-market, rising to thirteen percent in the cheapest decile and sixteen in the most expensive — both tails are priced on things we do not observe. On the right, by district: the worst is Mərdəkan, a garden-house suburb, where land value and build quality are not in our features; the k-means bonus confirms the periphery is hardest. Feature importances from the tree and the SVM weights tell the same story: area first, then geography.

Likely examiner questions

- **How did you compute the ±15% statistic?** For each misclassified test listing we check |price − 208,000| / 208,000 < 0.15 and report the share — it is in results.json as taskB_*_errors_within_15pct_of_threshold.
- **Would a different threshold (e.g. 75th percentile) help?** It would move the noisy band, not remove it, and would unbalance the classes. Testing other percentiles is listed as future work; the median was chosen because the statement suggests it and it balances the classes.

Slide 14 — Bonus studies: linear models cannot express area × location; a forest of our trees gains 0.008 RMSE

Speaker: Nihat · **time:** 0:50

Takeaway on slide: Each bonus reuses the core code: the forest is our tree with max_features; the SVR is Pegasos with a tube loss; Ridge is the derivation from the exam.

Script

Three bonus studies, each built on the core code rather than beside it. Ridge regression via the normal equations matches scikit-learn to machine precision, and a linear SVR trained by sub-gradient descent on the epsilon-insensitive loss — the regression twin of Pegasos — matches LinearSVR to four decimals. Both linear models stop at 0.30 RMSE on log-price against 0.23 for the tree: price formation is non-additive, area interacts with location, and a linear model cannot express that. Second, a random forest built from our own trees — thirty bootstrap trees with per-split feature subsampling — brings the tree from 0.229 down to 0.221, tracking scikit-learn's forest at 0.217; the curve shows test error falling as trees are added. Third, k-means on coordinates gives interpretable market segments and shows the periphery is where the regressor struggles.

Likely examiner questions

- **Why does your forest trail sklearn's by 0.0045?** Same reasons as the single tree — tie-breaking and feature-sampling order — compounded over 30 trees. The CV std of a single tree is 0.002, so the gap is small but probably real; we did not tune the forest.
- **Why does the SVR's best ε sit at the edge of the grid (0.2)?** A wider tube acts like extra regularisation on log-price noise; we swept only four values. Extending the grid is a cheap next step; the point of the bonus was matching the library's solver on the same loss.

Slide 15 — Reproducible by one command, tested against the library — and honest about what the models cannot see

Speaker: Nihat · **time:** 0:50

Takeaway on slide: Every claim on these slides is tied to a file in the repository and a number in results.json.

Script

Two things an examiner should be able to trust. First, reproducibility: one command runs ten sanity tests and regenerates every number and figure you have seen, from a fixed seed, in about ten minutes; three runs on our laptops were byte-identical. The tests are the interesting part — they pin our tree to scikit-learn's exact split on toy data, and our metrics to the library's to one-e-minus-nine. Second, limitations, stated plainly. These are asking prices, not transactions. The data is Baku only. The median split makes the classification label noisy at the cut. Exponentiating a log prediction estimates the median, not the mean, which biases the manat metrics. Our RFF was tuned over a small grid, and the exact RBF baseline used a subsample. And the free-text descriptions, which plausibly carry the luxury-tail signal, are unused.

Likely examiner questions

- **What is the single most important threat to validity?** Asking vs transaction prices: the target itself is seller behaviour. Every model here, including the baselines, inherits that; the comparison between implementations is unaffected, the absolute error is.
- **How do you know the runs are reproducible?** Fixed seeds for the split, the Pegasos sampler, the RFF draw and every sklearn baseline; results.json from three runs on two machines matched to the printed precision.

Slide 16 — The classical algorithms are small — the library's edge is engineering

Speaker: Nihat · **time:** 0:40

Script

To conclude. About 250 lines of careful NumPy reproduce scikit-learn's accuracy on 57 thousand real listings once the split objective and the sub-gradient schedule are implemented exactly; the remaining differences are tie-breaking and Cython, not statistics. We traced the residual error to the threshold band, the price tails and peripheral districts. Next steps would be text features from the descriptions, a group split by building, a quantile forest for price intervals, and a smearing correction for the manat metrics. Contributions were equal and are listed here and in the repository. Thank you — we are happy to take questions.

Likely examiner questions

- **If you had one more week, what would you do first?** Text features from the free-text descriptions — the luxury-tail error and the garden-house error both point at unobserved quality attributes, and the descriptions are where sellers write them.

- **Which part of the code would each of you defend?** Each member owns one module (see the table) and has commits in it; every member can also explain the overall pipeline, because run_all.py wires the modules together in one readable file.